

Slope

斜率优化

例题引入

▼ 「HNOI2008」玩具装箱

有 n 个玩具，第 i 个玩具价值为 w_i 。要求将这 n 个玩具排成一排，分成若干段。对于一段 $[l, r]$ ，它的代价为 $(r-l+1)^2 + w_l + w_r$ 。其中 C 是一个常量，求分段的最小代价。

。

朴素的 DP 做法

令 f_i 表示前 i 个物品，分若干段的最小代价。

状态转移方程：

其中 S_i 表示前 i 个数的和，即 $S_i = w_1 + w_2 + \dots + w_i$ 。

该做法的时间复杂度为 $O(n^2)$ ，无法解决本题。

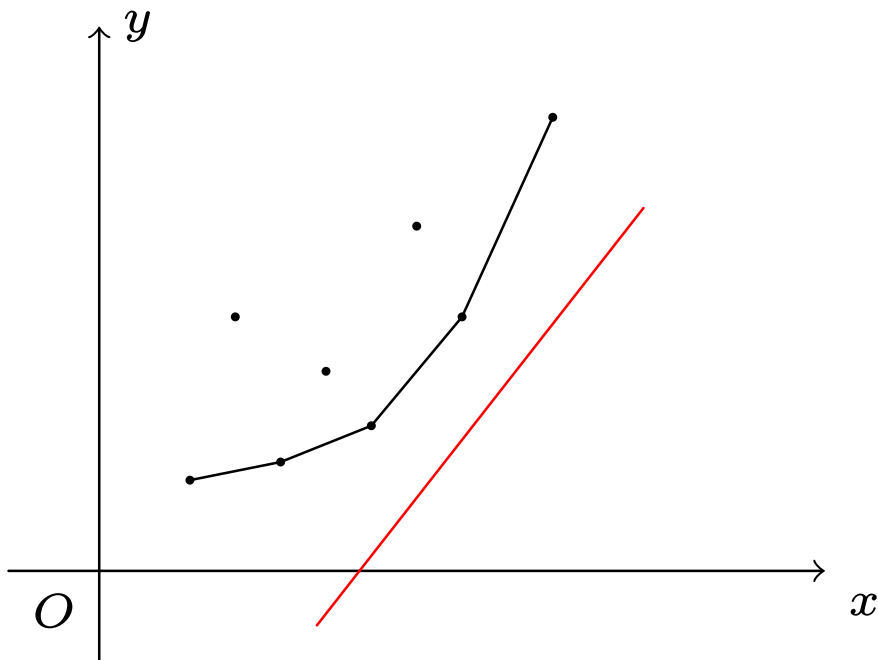
优化

考虑简化上面的状态转移方程式：令 $g_i = f_i + S_i^2$ ，则

将与 i 无关的移到外面，我们得到

考虑一次函数的斜截式，将其移项得到 $g_i - S_i^2 = f_{j-1} + S_{j-1}^2 + w_j + w_i$ 。我们将与 j 有关的信息表示为 $g_j - S_j^2$ 的形式，把同时与 j 有关的信息表示为 $2S_j$ ，把要最小化的信息（与 i 有关的信息）表示为 w_i ，也就是截距。具体地，设

则转移方程就写作 $g_i - S_i^2 = \min_{j < i} (g_j - S_j^2 + 2S_j w_i + w_i)$ 。我们把 $(S_j, g_j - S_j^2)$ 看作二维平面上的点，则 $2S_j$ 表示直线斜率， $g_j - S_j^2 + w_i$ 表示一条过 $(S_j, g_j - S_j^2)$ 的斜率为 $2S_j$ 的直线的截距。问题转化为了，选择合适的 j ，最小化直线的截距。



如图，我们将这个斜率为 k 的直线从下往上平移，直到有一个点在这条直线上，则有 $y = kx + b$ ，这时 b 取到最小值。算完，我们就把 (x, y) 这个点加入点集中，以做为新的 DP 决策。那么，我们该如何维护点集？

容易发现，可能让 b 取到最小值的点一定在下凸壳上。因此在寻找 b 的时候我们不需要枚举所有 n 个点，只需要考虑凸包上的点。而在本题中 k 随 x 的增加而递增，因此我们可以单调队列维护凸包。

具体地，设 k_i 表示过 (x_i, y_i) 和 (x_{i-1}, y_{i-1}) 的直线的斜率。考虑队列，维护的是下凸壳上的点。也就是说，对于 $i < j < k$ ，始终有 $k_i < k_j < k_k$ 成立。

我们维护一个指针 l 来计算 b 最小值。我们需要找到一个 i 的（特别地，当 $k_i = k$ 或者 $k_i > k$ 时要特别判断），这时就有 $y = kx + b$ ，即 i 是最优决策点。由于 k 是单调递增的，因此 i 的移动次数是均摊 $O(1)$ 的。

在插入一个点 (x, y) 时，我们要判断是否 $k_{i-1} < k_i < k_{i+1}$ ，如果不等式不成立就将 i 弹出，直到等式满足。然后将 (x, y) 插入到 i 队尾。

这样我们就将 DP 的复杂度优化到了 $O(n)$ 。

概括一下上述斜率优化模板题的算法：

1. 将初始状态入队。
2. 每次使用一条和 k 相关的直线去切维护的凸包，找到最优决策，更新 b 。
3. 加入状态 (x, y) 。如果一个状态（即凸包上的一个点）在 (x, y) 加入后不再是凸包上的点，需要在 (x, y) 加入前将其剔除。

接下来我们介绍斜率优化的进阶应用，将斜率优化与二分/分治/数据结构等结合，来维护性质不那么好（缺少一些单调性性质）的 DP 方程。

二分/CDQ/平衡树优化 DP

当我们在 (x, y) 这个点寻找最优决策时，会使用一个和 k 相关的直线去切我们维护的凸包。切到的点即为最优决策。

在上述例题中，直线的斜率随 x 单调变化，但是对于有些问题，斜率并不是单调的。这时我们需要维护凸包上的每一个节点，然后每次用当前的直线去切这个凸包。这个过程可以使用二分解决，因为凸包上相邻两个点的斜率是有单调性的。

▼ 玩具装箱 改

有 n 个玩具，第 i 个玩具价值为 v_i 。要求将这个玩具排成一排，分成若干段。对于一段 $[l, r]$ ，它的代价为 $(\sum_{i=l}^r v_i)^2 + c$ 。其中 c 是一个常量，求分段的最小代价。

。

本题与「玩具装箱」问题唯一的区别是，玩具的价值可以为负。延续之前的思路，令 f_i 表示前 i 个物品，分若干段的最小代价。

状态转移方程：

其中。

将方程做相同的变换

然而这时有两个条件不成立了：

1. 直线的斜率不再单调；
2. 每次加入的决策点的横坐标不再单调。

仍然考虑凸壳的维护。

在寻找最优决策点，也就是用直线切凸壳的时候，我们将单调队列找队首改为：凸壳上二分。我们二分出斜率最接近直线斜率的那条凸壳边，就可以找到最优决策。

在加入决策点，也就是凸壳上加一个点的时候，我们有两种方法维护。

第一种方法是直接用平衡树维护凸壳。那么寻找决策点的二分操作就转化为在平衡树上二分，插入决策点就转化为在平衡树上插入一个结点，并删除若干个被踢出凸壳的点。此方法思路简洁但实现繁琐。

下面介绍一种基于 [CDQ 分治](#) 的做法。

设 f_i 代表计算 f_i 。考虑：

- 我们先调用 f_{i-1} 算出 f_{i-1} 。然后我们对 $[l, r]$ 这个区间内的决策点建凸壳，然后使用这个凸壳去更新 f_i 。这时我们决策点集是固定的，不像之前那样边计算 DP 值边加入决策点，那么我们就可以把 f_{i-1} 先按照直线的斜率排序，然后就可以使用单调队列来计算 DP 值了。当然，也可以在静态凸壳上二分计算 DP 值。
- 对于 $[l, r]$ 中的每个点，如果它的最优决策的位置是在 $[l, r]$ 这个区间，在这一步操作中他就会被更新成最优答案。当执行完这一步操作时，我们发现 $[l, r]$ 中的所有点已经发挥了全部的作用，凸壳中他们存不存在已经不影响之后的答案更新。因此我们可以直接舍弃这个区间的决策点，并使用 f_{i-1} 解决右区间剩下的问题。

时间复杂度 $O(n \log n)$ 。

对比「玩具装箱」和「玩具装箱 改」，可以总结出以下两点：

- 二分/CDQ/平衡树等能够优化 DP 方程的计算，于一定程度上降低复杂度，但不能改变这个方程本身。
- DP 方程的性质会取决于数据的特征，但 DP 方程本身取决于题目中的数学模型。

小结

斜率优化 DP 需要灵活运用，其宗旨是将最优化问题转化为二维平面上与凸包有关的截距最值问题。遇到性质不太好的方程，有时需要辅以数据结构来加以解决，届时还请就题而论。

习题

- [「SDOI2016」征途](#)
- [「ZJOI2007」仓库建设](#)
- [「APIO2010」特别行动队](#)

- [「JSOI2011」柠檬](#)
 - [「Codeforces 311B」Cats Transport](#)
 - [「NOI2007」货币兑换](#)
 - [「NOI2019」回家路线](#)
 - [「NOI2016」国王饮水记](#)
 - [「NOI2014」购票](#)
-

本页面最近更新：2024/6/22 01:07:37, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[StudyingFather](#), [Enter-tainer](#), [lr1d](#), [luoguyuntianming](#), [Marcythm](#), [sshwy](#), [abc1763613206](#), [BackSlashDelta](#), [billchenchina](#), [Chrogeek](#), [ChungZH](#), [fps5283](#), [GavinZhengOl](#), [greyqz](#), [GreyTigerOler](#), [Henry-ZHR](#), [hsfzLZH1](#), [Kaiser-Yang](#), [Konano](#), [Luckyblock233](#), [mgt](#), [MrFoodinChina](#), [nanmenyangde](#), [ouuan](#), [wood3](#), [Xeonacid](#), [代建杉](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用